“Send Tip” flow — from the moment a fan clicks Tip IT! on a creator’s public profile to the instant the creator’s custodial Circle wallet is credited.

---

1 . High-level swim-lane / sequence diagram (Mermaid)

```
sequenceDiagram
    autonumber
    %%  UI actors
    participant Fan UI     as 🧑‍💻 Fan (React / Next.js) :fa-user-circle:
    participant CreatorUI  as 👩‍🎨 Creator Public Profile :fa-user-circle-o:
    %%  Platform + external
    participant TipAPI     as 🖥️ TipJar API (NestJS) :fa-server:
    participant CircleAPI  as 🔄 Circle Wallets & Payments API :fa-circle-o-notch:
    participant Bundler    as 📦 ERC-4337 Bundler (Pimlico) :fa-cubes:
    participant Paymaster  as 💳 Circle Paymaster :fa-credit-card:
    participant Polygon    as ⛓️ Polygon PoS Chain :fa-link:
    participant CreatorWal as 👛 Creator DCW (USDC) :fa-money:

    Fan UI->>CreatorUI: 1. Click **Tip IT!** (opens modal)
    Fan UI->>Fan UI: 2. Select amount, message, method
                    (Card / TipJar Wallet / External Wallet)

    alt A) Card / internal TipJar Wallet
        Fan UI->>TipAPI: 3a. `POST /tips` { amount, msg }
        TipAPI->>CircleAPI: 4a. `/transfers` (off-chain DCW→DCW)
        CircleAPI-->>TipAPI: 5a. success + tx id
        TipAPI-->>CreatorWal: 6a. credit USDC
        TipAPI-->>Fan UI: 7a. emit websocket “tip_confirmed”
    else B) External EOA (MetaMask + Paymaster)
        Fan UI->>TipAPI: 3b. `GET /paymaster-config`
        loop client-side
            Fan UI->>Fan UI: 4b-i. Create ERC-4337 UserOp
                            – prepare USDC permit (EIP-2612)
            Fan UI->>Bundler: 4b-ii. `eth_sendUserOperation`
            Bundler->>Paymaster: 5b. validate & quote gas in USDC
            Paymaster->>CircleAPI: 6b. charge USDC fee from Fan
            Bundler->>Polygon: 7b. submit UserOp → Tip Router SC
            Polygon-->>Bundler: 8b. Tx receipt (USDC → CreatorWal)
            Bundler-->>Fan UI: 9b. opHash / receipt
        end
        Fan UI-->>TipAPI: 10b. `POST /tips/confirm` opHash
        TipAPI-->>Fan UI: 11b. ack
    end
    Note over Fan UI,CreatorWal: Creator sees live overlay (socket)<br/>“🧑‍💻 tipped 5 USDC
– Thanks!”
```

> Legend :fa-…: are Font-Awesome icon hints (use directly in SVG / Figma exports or replace with <i class="">).
Replace Polygon with another USDC-native chain if desired.

---

2 . Key API / contract calls & payloads

#	Component	Sample call / payload highlights
3a	/tips (card / internal wallet)	POST /tips {creatorId, amountUsd: "5.00", msg} → returns tipId
4a	Circle Transfers API	POST /v1/transfers with source.walletId (fan DCW) → destination.walletId (creator DCW)
4b-ii	UserOperation fields	callData = TipRouter.tip(creator, amount); paymasterAndData = 0x… (quote)
5b	Paymaster validate	Pulls permit → transferFrom(fan, paymaster, gasCostUSDC)
7b	TipRouter SC	Emits TipSent(fan, creator, amount) (indexed)

---

3 . Front-end state hooks (excerpt)

```
// Zustand store snippet (see full plan)
const useTipStore = create<TipState>()((set) => ({
  sendTip: async ({creatorId, amount, method, msg}) => {
    if (method === 'CARD' || method === 'WALLET') {
      await fetch('/api/tips', {method:'POST', body: JSON.stringify({...})})
    } else {
      const cfg = await fetchJSON('/api/paymaster-config')
      const userOp = await buildUserOp(cfg, amount, creatorId)
      await bundler.sendUserOperation(userOp)
      await fetch('/api/tips/confirm', {method:'POST', body: JSON.stringify({opHash})})
    }
  }
}))
```

---

4 . Implementation checkpoints

1. Atomic bookkeeping – TipJar DB stores tipId ➜ creator wallet id + Circle tx id.

2. Web-socket updates – push to fan & OBS overlay (tip_confirmed, goal_progress).

3. Gas-sponsor fallback – if Paymaster unsupported chain, route to Card or TipJar Wallet.

4. Compliance – thresholds → trigger creator KYC before funds are withdrawable.

This diagram + notes should slot directly into your docs, hand-off to engineering, or be pasted in a Markdown file and rendered by Mermaid live preview. 🎉